arXiv:2312.00752v2 [cs.LG] 31 May 2024

Mamba: Linear-Time Sequence Modeling with Selective State Spaces

Albert Gu*[1] and Tri Dao*[2]

[1]Machine Learning Department, Carnegie Mellon University

[2]Department of Computer Science, Princeton University
agu@cs.cmu.edu, tri@tridao.me

Abstract

Foundation models, now powering most of the exciting applications in deep learning, are almost universally based on the Transformer architecture and its core attention module. Many subquadratic-time architectures such as linear attention, gated convolution and recurrent models, and structured state space models (SSMs) have been developed to address Transformers' computational inefficiency on long sequences, but they have not performed as well as attention on important modalities such as language. We identify that a key weakness of such models is their inability to perform content-based reasoning, and make several improvements. First, simply letting the SSM parameters be functions of the input addresses their weakness with discrete modalities, allowing the model to *selectively* propagate or forget information along the sequence length dimension depending on the current token. Second, even though this change prevents the use of efficient convolutions, we design a hardware-aware parallel algorithm in recurrent mode. We integrate these selective SSMs into a simplified end-to-end neural network architecture without attention or even MLP blocks (**Mamba**). Mamba enjoys fast inference (5× higher throughput than Transformers) and linear scaling in sequence length, and its performance improves on real data up to million-length sequences. As a general sequence model backbone, Mamba achieves state-of-the-art performance across several modalities such as language, audio, and genomics. On language modeling, our Mamba-3B model outperforms Transformers of the same size and matches Transformers twice its size, both in pretraining and downstream evaluation.

1 Introduction

Foundation models (FMs), or large models pretrained on massive data then adapted for downstream tasks, have emerged as an effective paradigm in modern machine learning. The backbone of these FMs are often *sequence models*, operating on arbitrary sequences of inputs from a wide variety of domains such as language, images, speech, audio, time series, and genomics (Brown et al. 2020; Dosovitskiy et al. 2020; Ismail Fawaz et al. 2019; Oord et al. 2016; Poli et al. 2023; Sutskever, Vinyals, and Quoc V Le 2014). While this concept is agnostic to a particular choice of model architecture, modern FMs are predominantly based on a single type of sequence model: the Transformer (Vaswani et al. 2017) and its core attention layer (Bahdanau, Cho, and Bengio 2015) The efficacy of self-attention is attributed to its ability to route information densely within a context window, allowing it to model complex data. However, this property brings fundamental drawbacks: an inability to model anything outside of a finite window, and quadratic scaling with respect to the window length. An enormous body of research has appeared on more efficient variants of attention to overcome these drawbacks (Tay, Dehghani, Bahri, et al. 2022), but often at the expense of the very properties that makes it effective. As of yet, none of these variants have been shown to be empirically effective at scale across domains.

Recently, structured state space sequence models (SSMs) (Gu, Goel, and Ré 2022; Gu, Johnson, Goel, et al. 2021) have emerged as a promising class of architectures for sequence modeling. These models can be interpreted as a combination of recurrent neural networks (RNNs) and convolutional neural networks (CNNs), with inspiration from classical state space models (Kalman 1960). This class of models can be computed very efficiently as either a recurrence or convolution, with linear or near-linear scaling in sequence length. Additionally, they have principled mechanisms for modeling long-range dependencies (Gu, Dao, et al. 2020) in certain data modalities, and have dominated benchmarks such as the Long Range

*Alphabetical by first name.

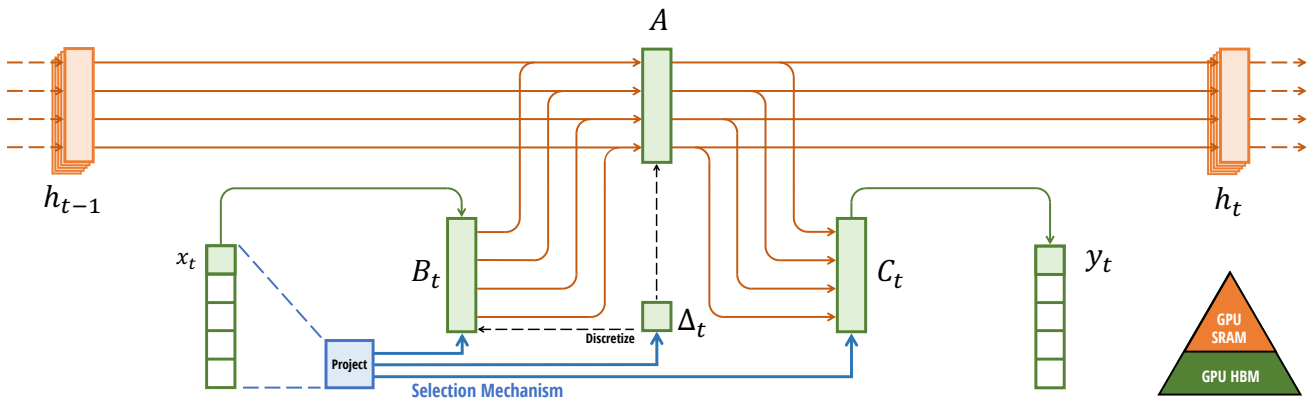


Figure 1: (**Overview**.) Structured SSMs independently map each channel (e.g. $D = 5$) of an input x to output y through a higher dimensional latent state h (e.g. $N = 4$). Prior SSMs avoid materializing this large effective state (DN, times batch size B and sequence length L) through clever alternate computation paths requiring time-invariance: the $(\Delta, \boldsymbol{A}, \boldsymbol{B}, \boldsymbol{C})$ parameters are constant across time. Our selection mechanism adds back input-dependent dynamics, which also requires a careful hardware-aware algorithm to only materialize the expanded states in more efficient levels of the GPU memory hierarchy.

1-dimensional function or sequence $x(t) \in \mathbb{R} \mapsto y(t) \in \mathbb{R}$ through an implicit latent state $h(t) \in \mathbb{R}^N$.

Concretely, S4 models are defined with four parameters $(\Delta, \boldsymbol{A}, \boldsymbol{B}, \boldsymbol{C})$, which define a sequence-to-sequence transformation in two stages.

$$h'(t) = \boldsymbol{A}h(t) + \boldsymbol{B}x(t) \quad \text{(1a)} \qquad h_t = \overline{\boldsymbol{A}}h_{t-1} + \overline{\boldsymbol{B}}x_t \quad \text{(2a)} \qquad \overline{\boldsymbol{K}} = (\boldsymbol{C}\overline{\boldsymbol{B}}, \boldsymbol{C}\overline{\boldsymbol{A}\boldsymbol{B}}, \dots, \boldsymbol{C}\overline{\boldsymbol{A}}^k\overline{\boldsymbol{B}}, \dots) \quad \text{(3a)}$$

$$y(t) = \boldsymbol{C}h(t) \quad \text{(1b)} \qquad y_t = \boldsymbol{C}h_t \quad \text{(2b)} \qquad y = x * \overline{\boldsymbol{K}} \quad \text{(3b)}$$

Discretization. The first stage transforms the "continuous parameters" $(\Delta, \boldsymbol{A}, \boldsymbol{B})$ to "discrete parameters" $(\overline{\boldsymbol{A}}, \overline{\boldsymbol{B}})$ through fixed formulas $\overline{\boldsymbol{A}} = f_A(\Delta, \boldsymbol{A})$ and $\overline{\boldsymbol{B}} = f_B(\Delta, \boldsymbol{A}, \boldsymbol{B})$, where the pair (f_A, f_B) is called a *discretization rule*. Various rules can be used such as the zero-order hold (ZOH) defined in equation (4).

$$\overline{\boldsymbol{A}} = \exp(\Delta\boldsymbol{A}) \qquad \overline{\boldsymbol{B}} = (\Delta\boldsymbol{A})^{-1}(\exp(\Delta\boldsymbol{A}) - \boldsymbol{I}) \cdot \Delta\boldsymbol{B} \tag{4}$$

Discretization has deep connections to continuous-time systems which can endow them with additional properties such as resolution invariance (Nguyen, Goel, et al. 2022) and automatically ensuring that the model is properly normalized (Gu, Johnson, Timalsina, et al. 2023; Orvieto et al. 2023). It also has connections to gating mechanisms of RNNs (Gu, Gulcehre, et al. 2020; Tallec and Ollivier 2018) which we will revisit in Section 3.5. However, from a mechanical point of view discretization can simply be viewed as the first step of the computation graph in the forward pass of an SSM. Alternate flavors of SSMs can bypass the discretization step and parameterize $(\overline{\boldsymbol{A}}, \overline{\boldsymbol{B}})$ directly instead (Zhang et al. 2023), which may be easier to reason about.

Computation. After the parameters have been transformed from $(\Delta, \boldsymbol{A}, \boldsymbol{B}, \boldsymbol{C}) \mapsto (\overline{\boldsymbol{A}}, \overline{\boldsymbol{B}}, \boldsymbol{C})$, the model can be computed in two ways, either as a **linear recurrence** (2) or a **global convolution** (3).

Commonly, the model uses the convolutional mode (3) for efficient parallelizable training (where the whole input sequence is seen ahead of time), and switched into recurrent mode (2) for efficient autoregressive inference (where the inputs are seen one timestep at a time).

Linear Time Invariance (LTI). An important property of equations (1) to (3) is that the model's dynamics are constant through time. In other words $(\Delta, \boldsymbol{A}, \boldsymbol{B}, \boldsymbol{C})$, and consequently $(\overline{\boldsymbol{A}}, \overline{\boldsymbol{B}})$ as well, are fixed for all time-steps. This property is